

TP1 - Registro de préstamos de la biblioteca

Cuestionario

1) ¿Qué es un TDA? ¿Por qué conviene separar el contrato (la interfaz) de la implementación?

Un Tipo de Dato Abstracto (TDA) es una forma de describir un dato a partir de las operaciones que se pueden realizar con él, sin especificar cómo esas operaciones están resueltas por dentro. En este TP, la interfaz `RegistroPrestamos` define el contrato (registrar, cantidad, obtener, padrones, prestamosDe, titulosMasPedidos) y `RegistroSobreArreglo` resuelve ese contrato usando un arreglo que crece a mano. Separar contrato de implementación permite que el resto del programa (`LectorPrestamos`, `Reporteador`) dependa únicamente de la interfaz, sin conocer la estructura interna. Esto hace posible cambiar la implementación en el futuro (por ejemplo, reemplazar el arreglo por otra estructura) sin modificar ni una línea del código que usa el registro, porque ese código nunca hizo referencia directa a la clase concreta.

2) ¿Qué es una invariante de clase? ¿Por qué conviene validarla en el constructor y no al principio de cada método?

Una invariante de clase es una condición que debe cumplirse siempre para que un objeto de esa clase sea considerado válido. En el record `Prestamo`, las invariantes son: que `socio`, `isbn` y `título` no sean nulos ni vacíos, que el `padrón` sea positivo, y que la fecha de devolución (si existe) no sea anterior a la de retiro. Validar estas condiciones en el constructor compacto garantiza que, una vez creado, un objeto `Prestamo` jamás pueda estar en un estado inconsistente, sin importar desde qué parte del programa se lo construya. Si en cambio se validara al principio de cada método, habría que repetir la misma lógica de validación en `cantidad()`, `diasDeAtraso()`, `multa()`, etc., aumentando la posibilidad de olvidarse de validar en algún método nuevo, y dejando abierta la posibilidad de que, entre la construcción del objeto y el primer método llamado, el objeto exista en un estado inválido.

3) La implementación duplica la capacidad del arreglo cada vez que se llena. Al registrar N préstamos, ¿cuántas copias de elementos se hacen en total? ¿Qué pasaría si en lugar de duplicar, el arreglo creciera de a un lugar por vez?

Con capacidad inicial 8 y duplicación, los redimensionamientos ocurren cuando la cantidad de elementos llega a 8, 16, 32, 64, ... En cada redimensionamiento se copian tantos elementos como tenía el arreglo en ese momento (8, luego 16, luego 32...). La suma total de copias realizadas al llegar a N elementos es menor que $2N$ (es una serie geométrica: $8 + 16 + 32 + \dots + N/2 < 2N$), por lo que el costo total de copiar, repartido entre las N inserciones, es $O(N)$ en total, es decir, $O(1)$ amortizado por inserción. Si en cambio el arreglo creciera de a un lugar por vez, cada inserción a partir de la novena necesitaría crear un arreglo nuevo y copiar todos los elementos existentes: la primera copia mueve 8 elementos, la siguiente 9, la siguiente 10, y así sucesivamente hasta N. La suma de esa serie es aproximadamente $N^2/2$, lo que hace que el costo total de las N inserciones sea $O(N^2)$, mucho más lento a medida que crece la cantidad de préstamos.

4) ¿Qué diferencia hay entre `==` y `equals` al comparar dos objetos `Prestamo`? ¿Por qué al usar un record el `equals` funciona sin escribirlo?

El operador `==` compara si dos referencias apuntan exactamente al mismo objeto en memoria. El método `equals` compara si el contenido de dos objetos es equivalente, aunque se trate de objetos distintos ubicados en posiciones de memoria diferentes. Por ejemplo, dos objetos `Prestamo` creados por separado pero con los mismos seis valores dan `== false` (son objetos distintos) pero `equals true` (representan el mismo préstamo). Un record de Java genera automáticamente, a partir de sus componentes declarados en

la firma, una implementación de equals (y de hashCode y toString) que compara campo por campo todos los componentes del record. Por eso no hace falta escribirlo a mano: el compilador ya sabe, a partir de la lista de componentes, qué campos hay que comparar para determinar si dos Prestamo son iguales.

5) ¿Por qué hay que usar `split(";", -1)` y no `split(";")` para partir cada línea del archivo? ¿Qué préstamos se romperían si se usara la segunda forma?

El método split de String, cuando recibe un límite negativo como segundo argumento, conserva todos los campos resultantes, incluyendo los vacíos que queden al final del arreglo. Sin ese segundo argumento (o con el límite en 0, que es el valor por defecto), Java descarta los elementos vacíos finales del resultado. En el archivo de préstamos, el campo fechaDevolucion puede venir vacío cuando el préstamo todavía no fue devuelto, y es justamente el último campo de la línea (después del último punto y coma no hay texto). Usando `split(";")` en una línea como "2026-03-11;40555;Diego Ruiz;9788478290499;El Lenguaje de Programacion C;", el resultado tendría solo 5 elementos en lugar de 6, porque el campo vacío final se descartaría. Eso haría que la validación de "se esperaban 6 campos" rechazara por error todas las líneas de préstamos pendientes (sin fecha de devolución), tratándolas como líneas inválidas cuando en realidad son correctas.